

Lab5实验报告

思考题

Thinking 5.1

Thinking 5.1 如果通过 `kseg0` 读写设备，那么对于设备的写入会缓存到 Cache 中。这是一种错误的行为，在实际编写代码的时候这么做会引发不可预知的问题。请思考：这么做 这会引发什么问题？对于不同种类的设备（如我们提到的串口设备和 IDE 磁盘）的操作会有差异吗？可以从缓存的性质和缓存更新的策略来考虑。

- 如果操作系统通过对 `kseg0` 的地址进行写入来操作外设，把一段新的数据写入了 `cache`，这一部分数据在下次触发 TLB Miss 时才会写回主存。但是在 Cache 未写回的时候，外设也会访问设备寄存器，从而使得设备运行错误。串讲也提到：

对设备物理内存的访问需要通过 `kseg1` 段。因为通过该段进行的读写不经过 TLB 映射，且不使用高速缓存。不使

用缓存的原因是设备物理内存处的数据不只由 CPU 决定，还和对应的外设的行为有关。而缓存只能记录 CPU 的

读写结果，无法在外设对数据进行修改时及时调整。

- 串口设备的通信频率比IDE磁盘的频率高得多，缓存会出现更多未更新为最新值的情况，从而错误出现得更频繁。

Thinking 5.2

Thinking 5.2 查找代码中的相关定义，试回答一个磁盘块中最多能存储多少个文件控制块？一个目录下最多能有多少个文件？我们的文件系统支持的单个文件最大为多大？

文件控制块大小 `FILE_STRUCT_SIZE` 为256，磁盘块大小 `BLOCK_SIZE` 为4096，一个磁盘块中最多存储 $\frac{4096}{256} = 16$ 个文件控制块。

`f_direct` 直接指针指向10个磁盘块，`f_indirect` 指向的磁盘块大小为4KB，磁盘号类型为 `uint32_t`，大小为4B，共可存储1024个磁盘号，前10个不用，指向1014个磁盘号，共1024个磁盘块。最多能有 $1024 \times 16 = 16K$ 个文件。一个磁盘块4KB，故最大单个文件为4MB。

Thinking 5.3

Thinking 5.3 请思考，在满足磁盘块缓存的设计的前提下，我们实验使用的内核支持的最大磁盘大小是多少？

`DISKMAX = 0x40000000`，即1GB空间。

Thinking 5.4

Thinking 5.4 在本实验中，`fs/serv.h`、`user/include/fs.h` 等文件中出现了许多宏定义，试列举你认为较为重要的宏定义，同时进行解释，并描述其主要应用之处。

- `PTE_DIRTY`：脏位，用来判断磁盘块对应虚拟地址的页是否经过修改，用在 `block_is_dirty()` 函数中，并用在 `unmap_block()` 中用来判断是否需要写回磁盘。
- `BLOCK_SIZE`：一个磁盘块的字节数，4096，4KB，与页面大小相等
- `BLOCK_SIZE_BIT`：一个磁盘块的总位数， 4096×8

- FILE2BLK: 一个磁盘块包含的文件控制块总数, 用于dir_lookup()等函数中进行每一个磁盘块中对文件的遍历。SECT_SIZE: 一个扇区包含的字节数, 512
- SECT2BLK: 一个磁盘块包含的扇区数, 在map_block(), unmap_block()中调用ide读写时使用, 因为ide读写函数是以扇区为单位的。
- DISKMAP、DISKMAX: 块缓存区域最低地址和块缓存占用的地址大小, 用于disk_addr()。

Thinking 5.5

在 Lab4“系统调用与 fork”的实验中我们实现了极为重要的 fork 函数。那么 fork 前后的父子进程是否会共享文件描述符和定位指针呢? 请在完成上述练习的基础上 编写一个程序进行验证。

- 会共享
- 程序参考了[这篇博客](#)

```

1  /*code*/
2  int r, fdnum, n;
3  char buf[200];
4  fdnum = open("/test", O_RDWR);
5  if((r = fork()) == 0) {
6      n = read(fdnum, buf, 4);
7      writef("[child] buffer is '%s'\n", buf);
8  } else {
9      n = read(fdnum, buf, 4);
10     writef("[father] buffer is '%s'\n", buf);
11 }
12
13 /*output*/
14 [father] buffer is 'BUAA'
15 [child] buffer is ' OS '

```

Thinking 5.6

Thinking 5.6 请解释 File, Fd, Filefd 结构体及其各个域的作用。比如各个结构体会在哪些过程中被使用, 是否对应磁盘上的物理实体还是单纯的内存数据等。说明形式自定, 要求简洁明了, 可大致勾勒出文件系统数据结构与物理实体的对应关系与设计框架。

```

1  struct File {
2      char f_name[MAXNAMELEN];    // 文件名
3      uint32_t f_size;            // 文件大小 (字节)
4      uint32_t f_type;           // 文件类型
5      uint32_t f_direct[NDIRECT]; // 直接指针
6      uint32_t f_indirect;        // 间接指针
7      struct File *f_dir;         // 父目录指针
8      char f_pad[FILE_STRUCT_SIZE - MAXNAMELEN - (3 + NDIRECT) * 4 -
9      sizeof(void *)]; // 补全空间
10 }

```

File 为文件控制块, 是物理实体。

- MOS 的文件系统是以根目录为根节点的树状数据结构。节点的数据结构由文件控制块 FCB (File 结构体) 定义。一个 File 结构体的大小为固定的 256 字节。在外存的块中存储

```

1 struct Fd {
2     u_int fd_dev_id;    //打开该文件时使用的设备id
3     u_int fd_offset;    //上次读写的偏移指针
4     u_int fd_omode;     //文件打开的模式
5 }

```

`Fd` 为文件描述符，是内存数据

```

1 struct Filefd {
2     struct Fd f_fd;    // 文件描述符
3     u_int f_fileid;    // 打开的文件的id
4     struct File f_file; // 打开的文件控制块
5 }

```

`Filefd` 为包含文件信息的文件描述符，是内存数据

- `open` 函数的返回值为一整数，称为文件描述符。文件描述符本质是对描述符表的索引，一个文件描述符对应一个 `Filefd` 结构体。该结构体在文件系统服务进程中创建，并通过 `ipc_send` 共享给其他进程。值得注意的是，在 `open` 等函数中存在将 `struct Fd*` 型指针强制转换为 `struct Filefd*` 型指针的操作。通过改变指针类型，实现了对相同数据的不同解释。

使用方式：

1. File 结构体

- **定义：**文件控制块（FCB），代表磁盘上的物理文件。
- **使用过程：**在文件创建、删除、读取、写入等操作中被使用。
- **对应关系：**直接对应磁盘上的物理文件，每个File结构体存储在磁盘的一个块中。

2. Fd 结构体

- **定义：**文件描述符，代表打开的文件在内存中的状态。。
- **使用过程：**在文件打开、关闭、读写等操作中被使用。
- **对应关系：**不直接对应磁盘上的物理实体，而是内存中的数据结构，用于管理打开的文件。

3. Filefd 结构体

- **定义：**包含文件信息的文件描述符，扩展了Fd结构体。
- **使用过程：**在文件打开、关闭、读写等操作中被使用，特别是用于在进程间共享文件信息。
- **对应关系：**同样不直接对应磁盘上的物理实体，而是内存中的数据结构，用于更全面地管理打开的文件。

文件系统数据结构与物理实体的对应关系与设计框架

- **树状结构：**文件系统以根目录为根节点，形成树状结构。
- **FCB与文件：**每个文件或目录由一个File结构体（FCB）表示，存储在磁盘上。
- **文件描述符与打开文件：**每个打开的文件由一个Fd或Filefd结构体表示，存储在内存中。
- **操作与结构体：**文件操作（如open、read、write）通过这些结构体来管理文件状态和数据。

Thinking 5.7

Thinking 5.7 图 5.9 中有多种不同形式的箭头，请解释这些不同箭头的差别，并思考我们的操作系统是如何实现对应类型的进程间通信的。

实心三角箭头代表着异步通信，即发生通信之后不等待目标进程的返回值，继续执行自己的代码。折线形的箭头代表同步通信，在这里就是用户线程向文件系统发出 `ipc_send` 请求必须在远处等待，直到收到文件系统的 `ipc_send` 返回值，两者才可以一起继续向后进行。

- 同步通信：用户向文件系统发送请求后，调用了 `ipc_recv()` 函数，它调用 `syscall_ipc_recv` 系统调用，进入阻塞态，等待文件系统发送给它 `syscall_ipc_send`，然后重新进入运行态，进入调度队列，可被调度。
- 异步通信：不需要特殊实现。

实验难点

一、外设控制模块

- 1. MMIO机制理解
 - 需掌握通过 `kseg1` 段访问设备寄存器的原理（非TLB映射/无缓存）
 - 系统调用 `sys_write_dev/sys_read_dev` 需严格检查地址范围和数据长度（1/2/4字节）
- 2. IDE磁盘驱动开发
 - 状态轮询：需实现 `wait_ide_ready()` 处理设备异步操作
 - 扇区操作：需按标准流程设置扇区号/模式/命令字（如 `MALTA_IDE_CMD_PIO_READ`）

二、文件系统实现

难点	关键挑战
磁盘布局设计	超级块/位图/数据块的物理分布管理，需处理 <code>struct Super</code> 的魔数校验
文件存储结构	直接指针 (<code>f_direct</code>) 与间接指针 (<code>f_indirect</code>) 的混合索引逻辑
块缓存机制	虚拟地址映射管理（ <code>disk_addr/map_block/unmap_block</code> ）
文件查找	<code>dir_lookup</code> 需遍历目录块并处理文件名匹配

三、进程间通信整合

- 1. IPC机制应用
 - 文件服务进程需通过 `ipc_recv/ipc_send` 处理请求
 - `fsipc_*` 系列函数需封装不同类型请求（如 `fsreq_remove`）
- 2. 描述符管理
 - `Fd` 与 `Filefd` 的结构差异及指针类型转换技巧
 - `open/read` 需协调描述符分配、权限检查和偏移量更新

四、实验工具链

- 磁盘镜像构建：需理解 `fs_format` 工具中 `write_directory/create_file` 的宿主机构建逻辑
- 调试复杂度：涉及用户态服务进程、设备驱动、IPC等多层交互，错误定位困难